Causal Analysis Library

Technical Reference & API Documentation

Cython-Optimised Pattern-Based Causal Discovery

February 28, 2026

Contents

1 Overview **2**

2 Mathematical Background **2**
- 2.1 Binary Entropy . . . 2
- 2.2 Weighted Entropy . . . 2
- 2.3 Average Weighted Entropy . . . 3
- 2.4 Causal Direction Rule . . . 3

3 API Reference **3**
- 3.1 `calculate_causal_history(seq_1, seq_2)` . . . 3
- 3.2 `sliding_xnor_comparison(p1, p2)` . . . 4
- 3.3 `generate_pattern_dictionary(G_input)` . . . 4
- 3.4 `calculate_contribution_analysis(patterns, X_in, Y_in)` . . . 4
- 3.5 `weighted_binary_entropy(ratio, count, seq_len, pattern_len)` . . . 5
- 3.6 `average_weighted_entropy(stats, seq_len)` . . . 5
- 3.7 `identify_causality(stats_x_y, stats_y_x, X, Y)` . . . 6
- 3.8 `run_causal_analysis(seq_x_bin, seq_y_bin)` . . . 6
- 3.9 `discrete(sequence, n_bins=2)` . . . 7
- 3.10 `generator(length, noise_intensity, a, b, phi)` . . . 7
- 3.11 `coupling_experiment(num_sequences, sequence_length, transients)` 8
- 3.12 `generate_coupled_data(b1, b2, initial_values)` . . . 8

4 Complete End-to-End Use Case **9**

5 Skew-Tent Map Helper (Internal) **10**
- 5.1 `c_skew_tent_map(x, b)` (cdef) . . . 10

6 Performance Notes **11**

7 Dependencies **11**

1 Overview

This library implements a **pattern-based causal discovery** framework for binary time series. The core idea follows from the *Minimum Uncertainty Principle*: the causal direction between two sequences X and Y is the one whose pattern-response relationship exhibits lower average weighted binary entropy. High regularity (low entropy) in how patterns in X precede changes in Y implies that X causally drives Y.

The pipeline consists of the following stages:

1. **Causal History Extraction** — identify sub-patterns in a source sequence that coincide with transitions in the target sequence.
2. **Pattern Dictionary Construction** — extract common sub-sequences across the causal history using XNOR-based sliding comparison.
3. **Contribution Analysis** — quantify how often each pattern in the source precedes a flip in the target.
4. **Causal Direction Inference** — compare average weighted entropies in both directions ($X \to Y$ and $Y \to X$) and return the direction with lower uncertainty.

All computationally intensive functions are implemented in **Cython** with typed memoryviews and compiler directives (`boundscheck=False`, `wraparound=False`) for near-C performance.

2 Mathematical Background

2.1 Binary Entropy

For a pattern with flip ratio r, the binary entropy is:

$$H_b(r) = -\big[r \log_2 r + (1-r)\log_2(1-r)\big]$$

$H_b(r) = 0$ when $r \in \{0, 1\}$ (perfectly deterministic), and $H_b(r)$ is maximised at $r = 0.5$ (maximum uncertainty).

2.2 Weighted Entropy

Each pattern p of length ℓ_p has total occurrence count c_p in a sequence of length N. Its weighted entropy is:

$$\tilde{H}(p) = \frac{c_p}{N - \ell_p + 1} \cdot H_b(r_p)$$

where r_p is the ratio of occurrences followed by a flip to total occurrences.

2.3 Average Weighted Entropy

For a pattern dictionary $\mathcal{P}$:

$$\overline{H} = \frac{1}{|\mathcal{P}|} \sum_{p \in \mathcal{P}} \tilde{H}(p)$$

2.4 Causal Direction Rule

$$\text{Direction} = \begin{cases} X \to Y & \text{if } \overline{H}_{X \to Y} < \overline{H}_{Y \to X} \\ Y \to X & \text{if } \overline{H}_{Y \to X} < \overline{H}_{X \to Y} \\ \text{Independence} & \text{if } \overline{H}_{X \to Y} = \overline{H}_{Y \to X} \end{cases}$$

3 API Reference

3.1 `calculate_causal_history(seq_1, seq_2)`

Purpose. Constructs the causal history dictionary $G_{X \to Y}$ by scanning the target sequence Y (`seq_1`) from left to right and recording the corresponding sub-pattern in the source sequence X (`seq_2`) each time a bit flip occurs in Y.

Inputs.

- `seq_1` — Target sequence Y (`str`, `list`, or `np.ndarray`).
- `seq_2` — Source sequence X (`str`, `list`, or `np.ndarray`).

Returns. `set` — Unique sub-strings from X that were active during each transition in Y.

Algorithm.

1. Scan Y from index $k = 1$ onwards.
2. When $Y[k] \neq Y[k-1]$ (a flip), append $X[\texttt{last_position} : k+1]$ to list G.
3. Update `last_position` to $k+1$ and continue.
4. Return the set of all collected segments.

```python
from cy_utils import calculate_causal_history

X = "10110010"
Y = "01100110"

G_xy = calculate_causal_history(Y, X)    # G_{X -> Y}
G_yx = calculate_causal_history(X, Y)    # G_{Y -> X}

print(G_xy)    # e.g., {'101', '10', '001'}
```

Listing 1: Example usage

3.2 `sliding_xnor_comparison(p1, p2)`

Purpose. Identifies common sub-sequences between two binary patterns using a sliding, position-by-position equality check (logically equivalent to XNOR). Only sub-sequences of length ≥ 2 are retained.

Inputs.

- `p1`, `p2` — Binary sub-patterns (`str`, `list`, or `np.ndarray`).

Returns. `set` — Common sub-sequences of length ≥ 2.

Algorithm. Let $n_1 \leq n_2$ be the pattern lengths (shorter is always aligned to longer). For each shift offset $s \in \{0, \ldots, n_2 - n_1\}$, a positional scan records contiguous matching characters. Any run of length ≥ 2 is added to the result set.

```python
from cy_utils import sliding_xnor_comparison

common = sliding_xnor_comparison("10110", "01101")
print(common)    # e.g., {'110', '101'}
```

Listing 2: Example usage

3.3 `generate_pattern_dictionary(G_input)`

Purpose. Constructs the pattern dictionary $P_{X \to Y}$ from the causal history $G_{X \to Y}$ by extracting all pairwise common sub-sequences via `sliding_xnor_comparison`.

Inputs.

- `G_input` — The causal history (`set` or `list` of strings).

Returns. `set` — The final pattern dictionary $P_{X \to Y}$. If no common patterns are found, returns `G_input` unchanged.

Complexity. $O(|G|^2)$ pair comparisons, each with $O(n^2)$ sliding work where n is pattern length.

```python
from cy_utils import generate_pattern_dictionary

G = {'101100', '10010', '1100'}
P = generate_pattern_dictionary(G)
print(P)     # e.g., {'101', '100', '10'}
```

Listing 3: Example usage

3.4 `calculate_contribution_analysis(patterns, X_in, Y_in)`

Purpose. Performs frequency and contribution analysis. For each pattern $p \in P_{X \to Y}$, counts how many times p appears in X and whether the corresponding window in Y contains a bit flip.

Inputs.

- `patterns` — Dictionary of patterns (`set` or `list`).
- `X_in` — Source sequence (`str`, `list`, or `np.ndarray`).
- `Y_in` — Target sequence (`str`, `list`, or `np.ndarray`).

Returns. `pd.DataFrame` with columns:

Column	Type	Description
`Change`	int	Occurrences of p in X followed by a flip in Y
`No-change`	int	Occurrences with no flip
`Ratio`	float	$R_{\text{flip}} = \texttt{Change}/(\texttt{Change} + \texttt{No-change})$

```python
from cy_utils import calculate_contribution_analysis

patterns = {'101', '10', '11'}
df = calculate_contribution_analysis(patterns, X="101100110", Y="
    011010100")
print(df)
```

Listing 4: Example usage

3.5 `weighted_binary_entropy(ratio, count, seq_len, pattern_len)`

Purpose. Computes the frequency-weighted binary entropy $\tilde{H}(p)$ for a single pattern.

Inputs.

- `ratio` — Flip probability $r_p \in [0, 1]$.
- `count` — Total occurrences of pattern p.
- `seq_len` — Length of the original sequence.
- `pattern_len` — Length of pattern p.

Returns. `float` — Weighted entropy value.

Notes.

- Returns 0.0 if $r_p \in \{0, 1\}$ (no uncertainty).
- Returns 0.0 if total sliding windows ≤ 0.

3.6 `average_weighted_entropy(stats, seq_len)`

Purpose. Computes the arithmetic mean of weighted binary entropies across all patterns in the dictionary.

Inputs.

- `stats` — `pd.DataFrame` returned by `calculate_contribution_analysis`.
- `seq_len` — Length of the original sequence.

Returns. `float` — Mean weighted entropy $\overline{H}$.

3.7 `identify_causality(stats_x_y, stats_y_x, X, Y)`

Purpose. Infers the causal direction between X and Y by comparing $\overline{H}_{X\to Y}$ and $\overline{H}_{Y\to X}$.

Inputs.

- `stats_x_y` — Contribution analysis DataFrame for $X \to Y$.
- `stats_y_x` — Contribution analysis DataFrame for $Y \to X$.
- `X`, `Y` — Original sequences (used to determine length).

Returns. `tuple`:

- `str` — One of `'X -> Y'`, `'Y -> X'`, `'Independence'`, `'Undetermined'`.
- `float` — $\overline{H}_{X\to Y}$.
- `float` — $\overline{H}_{Y\to X}$.

3.8 `run_causal_analysis(seq_x_bin, seq_y_bin)`

Purpose. Convenience function that executes the *complete* four-stage causal discovery pipeline on a pair of binary sequences.

Inputs.

- `seq_x_bin` — Discretised binary array for X.
- `seq_y_bin` — Discretised binary array for Y.

Returns. Same as `identify_causality`: `(direction, H_xy, H_yx)`.

```python
import numpy as np
from cy_utils import discrete, run_causal_analysis

# Synthetic data
rng = np.random.default_rng(42)
X_cont = np.cumsum(rng.normal(size=500))
Y_cont = 0.7 * X_cont + rng.normal(scale=0.3, size=500)

X_bin = discrete(X_cont, n_bins=2)
Y_bin = discrete(Y_cont, n_bins=2)

direction, h_xy, h_yx = run_causal_analysis(X_bin, Y_bin)
print(f"Direction : {direction}")
print(f"H(X->Y)   : {h_xy:.4f}")
print(f"H(Y->X)   : {h_yx:.4f}")
```

Listing 5: Full pipeline in one call

3.9 `discrete(sequence, n_bins=2)`

Purpose. Discretises a continuous numerical time series into n uniform bins, mapping each value to an integer label in $\{0, 1, \ldots, n_\text{bins} - 1\}$.

Inputs.

- `sequence` — 1-D `np.ndarray` of continuous values.
- `n_bins` — Number of discrete levels (default: `2`).

Returns. `np.ndarray` of `int` in $[0, n_\text{bins} - 1]$.

Notes. If `sequence` has zero range (constant signal), returns an array of zeros. Boundary values are clipped to $n_\text{bins} - 1$.

```python
import numpy as np
from cy_utils import discrete

signal = np.array([0.1, 0.5, 0.9, 0.3, 0.7])
binary = discrete(signal, n_bins=2)
print(binary)  # e.g., [0 1 1 0 1]
```

Listing 6: Binarising a signal

3.10 `generator(length, noise_intensity, a, b, phi)`

Purpose. Generates synthetic bivariate time series from a coupled first-order Autoregressive (AR) model:

$$Y[t] = b \cdot Y[t-1] + \epsilon_Y[t]$$
$$X[t] = a \cdot X[t-1] + \phi \cdot Y[t-1] + \epsilon_X[t]$$

where $\epsilon \sim \mathcal{N}(0, \sigma^2)$ and ϕ controls the coupling strength from Y to X.

Parameters.

Arg	Type	Default	Description
`length`	int	1000	Number of time steps
`noise_intensity`	float	0.01	Noise std. dev. σ
`a`	float	0.8	AR coefficient for X
`b`	float	0.8	AR coefficient for Y
`phi`	float	0.0	Coupling strength $Y \to X$

Returns. `tuple[np.ndarray, np.ndarray]` — (X, Y).

3.11 `coupling_experiment(num_sequences, sequence_length, transients)`

Purpose. Ensemble simulation over a range of coupling strengths $\phi \in [0, 0.95]$ (step 0.05). Discards initial `transients` steps to remove start-up artefacts.

Returns. `dict` — Keys are ϕ values; values are {`'X': ndarray, 'Y': ndarray`} with shape `(num_sequences, sequence_length)`.

```python
from cy_utils import coupling_experiment, discrete,
    run_causal_analysis

data = coupling_experiment(num_sequences=100, sequence_length
    =500, transients=200)

phi_val = 0.5
X_raw = data[phi_val]['X'][0]    # first realisation
Y_raw = data[phi_val]['Y'][0]

X_bin = discrete(X_raw)
Y_bin = discrete(Y_raw)

direction, h_xy, h_yx = run_causal_analysis(X_bin, Y_bin)
print(direction)    # expected: 'Y -> X' since phi > 0
```

Listing 7: Running coupling experiment

3.12 `generate_coupled_data(b1, b2, initial_values)`

Purpose. Simulates a **Master-Slave coupled Skew-Tent map** system for a range of coupling coefficients $\eta \in [0.0, 0.9]$:

$$M[t] = T_1\big(M[t-1]\big)$$
$$S[t] = (1-\eta)\, T_2\big(S[t-1]\big) + \eta\, M[t]$$

where $T_b(x)$ is the Skew-Tent map with parameter b.

Parameters.

Arg	Type	Default	Description
`b1`	float	0.65	Skewness for Master map T_1
`b2`	float	0.47	Skewness for Slave map T_2
`initial_values`	dict	None	Optional fixed initial conditions

Returns. `dict` — Keyed by η (string), each containing `'X'`, `'Y'`, and `'initial_conditions'`.

Notes. 2000 trials per η; 500 transient steps discarded; 1500 time steps retained per trial.

```python
from cy_utils import generate_coupled_data, discrete,
    run_causal_analysis

data = generate_coupled_data()

# Analyse a single trial at eta = 0.5
trial_X = data['0.5']['X'][0]
trial_Y = data['0.5']['Y'][0]

X_bin = discrete(trial_X)
Y_bin = discrete(trial_Y)

direction, h_xy, h_yx = run_causal_analysis(X_bin, Y_bin)
print(f"Inferred direction: {direction}")  # expected: 'Y -> X'
```

Listing 8: Chaotic map experiment

4 Complete End-to-End Use Case

The following example demonstrates the full workflow from raw continuous signals to a causal direction verdict.

```python
import numpy as np
from cy_utils import (
    discrete, calculate_causal_history,
    generate_pattern_dictionary, calculate_contribution_analysis,
    identify_causality
)

#         1. Generate or load continuous data


np.random.seed(0)
N = 800
Y_cont = np.cumsum(np.random.normal(0, 1, N))          #
    independent driver
X_cont = 0.6 * np.roll(Y_cont, 1) + np.random.normal(0, 0.5, N)
    # lagged Y

#         2. Discretise


X_bin = discrete(X_cont, n_bins=2)
Y_bin = discrete(Y_cont, n_bins=2)

X_str = "".join(map(str, X_bin))
Y_str = "".join(map(str, Y_bin))

#         3. Causal histories
```

```
G_xy = calculate_causal_history(Y_str, X_str)   # G_{X -> Y}
G_yx = calculate_causal_history(X_str, Y_str)   # G_{Y -> X}

#         4. Pattern dictionaries


P_xy = generate_pattern_dictionary(G_xy)
P_yx = generate_pattern_dictionary(G_yx)

#         5. Contribution analysis


stats_xy = calculate_contribution_analysis(P_xy, X_str, Y_str)
stats_yx = calculate_contribution_analysis(P_yx, Y_str, X_str)

print("X->Y pattern stats:")
print(stats_xy.head())

#         6. Infer direction


direction, h_xy, h_yx = identify_causality(stats_xy, stats_yx,
    X_str, Y_str)
print(f"\nInferred causal direction : {direction}")
print(f"  H_avg (X->Y) = {h_xy:.5f}")
print(f"  H_avg (Y->X) = {h_yx:.5f}")
```

Listing 9: End-to-end causal discovery

Expected output (may vary with random seed):

```
Inferred causal direction : Y -> X
  H_avg (X->Y) = 0.21347
  H_avg (Y->X) = 0.18920
```

5 Skew-Tent Map Helper (Internal)

5.1 `c_skew_tent_map(x, b)` (cdef)

Purpose. Pure-C internal function computing one iteration of the Skew-Tent map:

$$T_b(x) = \begin{cases} x/b & \text{if } 0 \le x < b \\ (1-x)/(1-b) & \text{if } b \le x < 1 \\ 0 & \text{otherwise} \end{cases}$$

This function is not accessible from Python; it is called only within `generate_coupled_data`.

6 Performance Notes

- Compiler directives `boundscheck=False`, `wraparound=False`, and `initializedcheck=False` remove Python-level index safety checks for maximum throughput.
- All inner loops operate on typed C `uint8` memory-views (`cnp.uint8_t[:]`) rather than Python strings, avoiding repeated object allocation.
- The `_ensure_uint8` helper transparently converts `str`, `list`, and `np.ndarray` inputs into a consistent byte view.
- For very long sequences ($N > 10{,}000$) or large pattern dictionaries, the $O(|P| \cdot N)$ contribution analysis remains the dominant cost.

7 Dependencies

Package	Purpose
`numpy`	Array operations, random number generation
`pandas`	Contribution analysis result formatting
`cython`	C-extension compilation